

Análisis Crítico del Manifiesto ACIDE: Evaluación de Patrones Arquitectónicos para Monolitos Modulares con CQRS y DDD

Gonzalo Enrique Gutiérrez Castillo

Área de Digitales – ACIDE

Arequipa, Perú

gonzalog@acideperu.org

gonzalogut.99@gmail.com

Registro de Disidencia Fundamentada (Punto 3.3.D)

Resumen—El Manifiesto de Arquitectura Evolutiva y Gestión de Activos Digitales (ACIDE) establece un estándar técnico para la construcción de módulos reutilizables en un ecosistema de monolito modular con Clean Architecture, CQRS y DDD en .NET. Este paper realiza una evaluación imparcial del Manifiesto, reconociendo sus contribuciones en gobernanza, reutilización vía el modelo Snapshot & Fork, y separación de responsabilidades. Simultáneamente, identifica ocho antipatrones que emergen de aplicar reglas de aislamiento diseñadas para sistemas distribuidos a un monolito que comparte proceso, base de datos y transacción. El análisis se fundamenta en la literatura canónica de DDD, Clean Architecture, CQRS y patrones de monolitos modulares, incluyendo documentación oficial de Microsoft, trabajos de Evans, Vernon, Newman, Fowler y Martin. Se proponen tres enmiendas concretas —separación asimétrica CQRS, dependencias explícitas direccionales, y comunicación inter-modular atómica— que preservan los objetivos del Manifiesto sin sacrificar la integridad técnica del sistema.

Index Terms—Clean Architecture, CQRS, DDD, monolito modular, patrones arquitectónicos, gobernanza técnica, antipatrones, .NET

I. INTRODUCCIÓN

La gestión de activos de software reutilizables es un desafío reconocido en la ingeniería de software empresarial [20]. En consultoras que desarrollan soluciones para múltiples clientes, la capacidad de extraer, catalogar y reutilizar módulos funcionales tiene un impacto directo en el *time-to-market* y la calidad [25].

El Manifiesto ACIDE [1] aborda este desafío proponiendo una “Constitución Técnica” que estandariza la estructura, comunicación y gobernanza de módulos en un ecosistema .NET. Su visión —transformar código de proyecto en “activos de negocio” reutilizables— es estratégicamente sólida y se alinea con la literatura de *Software Product Lines* [21].

Sin embargo, un estándar técnico obligatorio requiere precisión quirúrgica en sus reglas. Una regla mal calibrada no solo falla en su objetivo, sino que genera complejidad accidental [19] que erosiona la productividad que busca proteger.

Este paper evalúa el Manifiesto de forma imparcial: reconoce sus fortalezas en la Sección III, identifica antipatrones en la Sección IV, analiza tres observaciones técnicas críticas

en las Secciones V–VII, y propone enmiendas concretas en la Sección IX.

El análisis se realiza bajo el derecho de Registro de Disidencia Fundamentada (Punto 3.3.D del Manifiesto).

II. CONTEXTO ARQUITECTÓNICO

Antes de evaluar el Manifiesto, es fundamental establecer el contexto real del sistema ACIDE:

- **Monolito modular** [7]: un solo proceso desplegable con separación lógica interna en módulos.
- **Una sola base de datos relacional**: PostgreSQL o SQL Server, compartida por todos los módulos.
- **EF Core** como ORM: con *change tracking* y transacciones ACID.
- **CQRS *stateful* sin Event Sourcing**: separación de commands y queries, sin *event store*.
- **Modelo Snapshot & Fork**: los módulos se copian del Garaje al proyecto. No son dependencias vivas ni independientemente desplegables.

Esta distinción es crítica. Como establece Newman [7], un monolito modular y un sistema de microservicios tienen propiedades arquitectónicas fundamentalmente distintas. Las reglas de aislamiento que son *necesarias* en microservicios (donde cada servicio tiene su propia base de datos y proceso) generan costos sin beneficio en un monolito que ya comparte estos recursos.

Martin [4] en *Clean Architecture* es explícito: “A good architecture allows major decisions to be deferred”. El Manifiesto acierta en la estructura de capas (Domain, Application, Infrastructure, API), pero sus reglas de comunicación inter-modular toman una decisión prematura: tratan módulos co-desplegados como si fueran servicios independientes.

III. FORTALEZAS DEL MANIFIESTO

Es importante reconocer que el Manifiesto ACIDE contiene decisiones bien fundamentadas:

III-A. Visión Estratégica (Sección 1)

La transformación de “código de proyecto” a “activo de negocio” es una aplicación práctica de los principios de *Software Product Lines* [21]. La metáfora del “Garaje” como repositorio de módulos base es clara y operativa.

III-B. Modelo Snapshot & Fork (Sección 3)

Este modelo resuelve elegantemente el problema de dependencias vivas entre proyectos. Al copiar en vez de referenciar, cada proyecto local es autónomo. Esto elimina el riesgo de que una actualización en el Garaje rompa sistemas en producción, un problema real documentado por Humble y Farley [22].

III-C. Estructura de Capas (Sección 4)

La separación en Domain, Application, Infrastructure y API es consistente con Clean Architecture [4] y la Arquitectura Hexagonal de Cockburn [16]. La regla de que Domain no importe infraestructura (“Dependencia Cero”) es un principio sólido de DDD [2].

III-D. Gobernanza (Secciones 3.3 y 6)

El protocolo de auditoría, consenso documentado, ventana de revisión de 48h/1 semana, y el Registro de Disidencia Fundamentada son mecanismos maduros de gobernanza técnica. El criterio de “Extraibilidad” (Punto 6.2) es una heurística útil para validar la calidad de un módulo.

III-E. Política de IA (Sección 7)

La prohibición de código “caja negra” y la exigencia de que el desarrollador entienda y pueda mantener todo código generado por IA es una postura sensata y alineada con las mejores prácticas de la industria.

IV. ANTIPATRONES IDENTIFICADOS

A pesar de sus fortalezas, el Manifiesto introduce antipatrones al importar reglas de sistemas distribuidos. Los clasificamos usando la taxonomología de Brown et al. [23]:

IV-A. AP-1: Distributed Monolith

Definición: Aplicar reglas de aislamiento de microservicios dentro de un monolito que comparte proceso, base de datos y transacción [7].

Newman acuñó el término y advierte: “A *distributed monolith* is a system that consists of multiple services, but for whatever reason the entire system has to be deployed together. [...] It has all the disadvantages of a distributed system, and the disadvantages of a single-process monolith, without having enough upsides of either” [6].

En ACIDE, los módulos no son servicios independientes: comparten DbContext, transacción y proceso. Tratarlos como compartimentos estancos comunicados solo por eventos impone los costos del aislamiento distribuido sin obtener sus beneficios (escalabilidad independiente, despliegue autónomo, tolerancia a fallos parcial).

IV-B. AP-2: Middle Man / Pass-through Layer

Definición: Una capa intermedia que no agrega valor y solo reenvía llamadas [9].

Cuando las queries pasan por interfaces de repositorio en Domain que solo delegan a Infrastructure, se tiene: query handler → interfaz → repositorio → EF Core. Tres indirectrices para un SELECT. Fowler identifica esto como el *smell* “Middle Man” [9].

IV-C. AP-3: Header Interface

Definición: Interfaces con decenas de métodos específicos que no representan una abstracción coherente [8].

Al obligar que todas las queries pasen por interfaces en Domain, estas crecen con métodos de un solo uso: GetByIdWithIncludes, GetPaginatedByStatus, SearchByNameAndDate. Martin lo identifica como una violación del *Interface Segregation Principle* [5].

IV-D. AP-4: Leaky Abstraction Inversa

Definición: Una abstracción que modela implícitamente las capacidades de la tecnología que pretende ocultar [18].

Las interfaces de consulta en Domain terminan exponiendo conceptos de EF Core (paginación, Include, sorting). La abstracción “pura” está moldeada por el ORM.

IV-E. AP-5: Theatrical Decoupling

Definición: Desacoplamiento que existe en el código pero no en la realidad operativa.

Los módulos ACIDE siempre se copian juntos, comparten la misma base de datos y se despliegan en el mismo proceso. El aislamiento total existe en la estructura de carpetas pero no en el sistema real. Como advierte Tilkov [17]: “If modules are always deployed together, always share the same database, and always change together, they are not independent—they are one system with internal structure”.

IV-F. AP-6: Eventos sin Infraestructura de Eventos

Los domain events vía MediatR se procesan en la misma transacción (*in-process*). Esto no es comunicación asíncrona real: si el handler del evento falla, la transacción completa hace rollback. Como documenta Grzybek [14]: “In-process domain events using MediatR are synchronous by nature. They execute within the same transaction scope, which means they couple the modules transactionally”.

IV-G. AP-7: Composición Imposible

El Punto 5.7.B propone composición por interfaces (ICalculadorDeImpuestos), pero:

- Si usa tipos primitivos, pierde el valor de DDD [2].
- Si usa tipos del Domain de otro módulo, viola el cross-import.
- Si duplica tipos, arriesga la “Divergencia Salvaje” (Punto 3.5).
- Para que otro módulo implemente la interfaz, necesita referenciar el assembly donde está definida.

No existe código compilable que satisfaga todas las reglas simultáneamente.

IV-H. AP-8: DTOs Internos sin Rol Definido

Application contiene “DTOs internos” (Punto 4.2.B), pero el Manifiesto no define su rol en la comunicación inter-modular. En la arquitectura actual de ACIDE, estos DTOs ya funcionan como **contratos públicos validados**: cada Command DTO tiene un Validator (FluentValidation) que protege la frontera del módulo, sea que el comando venga de la API o de otro handler. El Manifiesto debería reconocer explícitamente este patrón —Command DTO + Validator como contrato inter-modular— en vez de clasificarlos como “internos”, lo cual contradice su uso real y oscurece un mecanismo que ya resuelve el problema de comunicación tipada entre módulos.

V. OBSERVACIÓN 1: SEPARACIÓN DE LECTURAS Y ESCRITURAS

V-A. Lo que establece el Manifiesto

La separación Domain→Interface→Infrastructure es uniforme para todas las operaciones. No distingue entre lecturas y escrituras.

V-B. Análisis del Problema

CQRS [11] se fundamenta en una asimetría esencial: las **escrituras** protegen invariantes de negocio y requieren el modelo de dominio completo; las **lecturas** devuelven datos planos sin mutar estado. Esta asimetría debería reflejarse en la arquitectura.

Contradicción con la guía oficial de Microsoft

La documentación oficial de Microsoft para .NET [12] establece:

“This guide suggests using DDD patterns only in the transactional/updates area of your microservice (that is, as triggered by commands). Queries can follow a simpler approach.”

Y más específicamente sobre queries:

“If you’re using the CQS/CQRS architectural pattern, the initial queries are performed by side queries out of the domain model, performed by simple SQL statements using Dapper. This approach is much more flexible than repositories.”

Jimmy Bogard, creador de MediatR (la librería que ACIDE usa para CQRS), lo confirma: *“I’m really not a fan of repositories, mainly because they hide the important details of the underlying persistence mechanism. Going CQRS meant that we didn’t really have a need for repositories any more”* [13].

Vernon [3] en *Implementing Domain-Driven Design* establece que los repositorios son un patrón de *escritura*: *“Repositories are used to persist and reconstitute Aggregates”*. Las queries de lectura operan sobre *projections* o vistas, no sobre aggregates.

Greg Young [11], quien formalizó CQRS, es explícito: *“The query side is not a domain. There is no behavior to it. Queries are a reporting concern, not a domain concern”*.

V-C. Respuesta al Argumento de Portabilidad de ORM

El argumento “si cambiamos de ORM, la interfaz nos protege” no resiste análisis en el contexto de ACIDE:

1. **Probabilidad:** EF Core es el ORM oficial de Microsoft con soporte de largo plazo. En una consultora .NET, la probabilidad de migrar es prácticamente nula.
2. **La abstracción no protege:** Si se cambia de ORM, se reescribe toda la capa de Infrastructure. La interfaz no ahorra ese trabajo [8].
3. **Sin abstracción, la migración es más granular:** Se puede migrar query por query. eShopOnContainers [12] usa EF Core para commands y Dapper para queries en el mismo proyecto.
4. **YAGNI:** Martin [5] y Beck [24] coinciden: no se debe pagar el costo de una abstracción para un escenario hipotético. *“You Aren’t Gonna Need It”*.

VI. OBSERVACIÓN 2: AISLAMIENTO TOTAL ENTRE MÓDULOS

VI-A. Lo que establece el Manifiesto

Punto 4.3 y 5.2: “Prohibido el Cross-Import. Un módulo no puede importar archivos internos de otro.” Los módulos deben ser compartimentos estancos.

VI-B. Análisis del Problema

VI-B1. Pérdida de Integridad Referencial: En un monolito con una sola base de datos, si Matricula tiene un FK a Alumno, EF Core necesita conocer el tipo Alumno para configurar la relación. Sin cross-import, las opciones son:

- Usar Guid sin constraint real → pierde integridad referencial.
- No tener navigation properties → pierde Include(), JOINS y type-safety.
- Duplicar la entidad → arriesga “Divergencia Salvaje”.

Grzybek [14] en su implementación de referencia de Modular Monolith con DDD resuelve esto permitiendo que un módulo referencie los *contratos públicos* de otro módulo, manteniendo *internal* los repositorios e infraestructura.

VI-B2. La Composición del Punto 5.7.B no Compila:

Contradicción lógica interna

Sea Módulo A que define ICalculadorDeImpuestos y Módulo B que lo implementa:

- B necesita referenciar A para conocer la interfaz → esto es un cross-import.
- Si la interfaz va a shared/, A ya no la “posee” → viola Propiedad del Dato (5.5).
- Si la interfaz usa decimal en vez de Money (Value Object), pierde DDD.
- Si la interfaz usa Money de A, B necesita importar de A → cross-import.

No existe solución que satisfaga todas las restricciones simultáneamente.

VI-B3. *Evidencia de la Industria*: Evans [2] en el *Blue Book* no propone aislamiento total entre *Bounded Contexts* dentro del mismo proceso. Define tres tipos de relación: *Shared Kernel*, *Customer-Supplier*, y *Conformist*. Todos permiten referencias cruzadas controladas.

Vernon [3] distingue explícitamente entre *Bounded Contexts* en el mismo proceso y en procesos separados: “*When two Bounded Contexts reside in the same process, they can communicate through simple method invocations or direct object references. The need for full isolation applies primarily when contexts are deployed independently*”.

VII. OBSERVACIÓN 3: COMUNICACIÓN INTER-MODULAR

VII-A. Lo que establece el Manifiesto

Punto 5.6: Tres opciones de comunicación — eventos de dominio, interfaces compartidas en *shared*, u orquestación en *Application* (“sin permitir que un módulo conozca a otro”).

VII-B. Análisis del Problema

VII-B1. *Atomicidad Transaccional*: En un sistema CQRS stateful sin Event Sourcing, las operaciones cross-module atómicas son frecuentes. Ejemplo: crear un Año Académico con sus Grados y Secciones requiere una sola transacción.

Las tres opciones del Manifiesto fallan en atomicidad:

1. **Eventos de dominio**: Vía MediatR son *in-process* y síncronos [13]. Se ejecutan en la misma transacción, lo que acopla transaccionalmente los módulos. Si un handler falla, todo el rollback afecta a todos los módulos. Esto es peor que una referencia directa: el acoplamiento es implícito.
2. **Interfaces en shared**: Requieren resolver las contradicciones del Punto 5.7.B (AP-7).
3. **Orquestación en Application**: El Manifiesto dice que *Application* “solo conoce al *Domain*”. ¿Cómo orquesta módulos que no puede importar?

VII-B2. *Solución que ya Existe en el Sistema*: La arquitectura actual de ACIDE ya resuelve este problema con un patrón documentado por Jovanović [15]:

- Cada **Command** tiene un **DTO** (record) como contrato tipado.
- Cada **Command** tiene un **Validator** (FluentValidation) como frontera.
- El método **Execute** contiene lógica sin llamar a *SaveChangesAsync*.
- Un handler orquestador llama a *Execute* de múltiples módulos y controla una sola transacción.

Este patrón es equivalente al *Application Service* de Vernon [3]: “*Application Services coordinate the activity of Aggregates and may cross Bounded Context boundaries when orchestrating use cases*”.

VIII. DISCUSIÓN: LA FALACIA DEL AISLAMIENTO PREMATURO

El hilo conductor de los ocho antipatrones es la **aplicación prematura de reglas de aislamiento**. Fowler [10] advierte contra esto con el patrón “Monolith First”:

“*Don’t start a new project with microservices, even if you’re sure your application will be big enough to make it worthwhile. [...] Almost all the successful microservice stories have started with a monolith that got too big and was broken up.*”

El Manifiesto ACIDE reconoce correctamente que el sistema es un monolito modular (no microservicios). Sin embargo, impone reglas de aislamiento de microservicios. Esto genera lo que Newman [7] denomina “*the worst of both worlds*”: los costos de coordinación de un sistema distribuido con las limitaciones de un monolito.

Tabla I resume las diferencias entre las reglas apropiadas para cada contexto:

Tabla I
REGLAS DE AISLAMIENTO: MICROSERVICIOS VS. MONOLITO MODULAR

Aspecto	Microservicios	Monolito Modular
Base de datos	Una por servicio	Compartida
Cross-import	Imposible (procesos separados)	Controlado (mismo proceso)
Transacciones	Sagas / eventual consistency	ACID nativas
Comunicación	HTTP / mensajería	Métodos directos / MediatR
Deploy	Independiente	Conjunto
FKs cross-module	No aplica	Posible y deseable

Grzybek [14], en su implementación de referencia de *Modular Monolith with DDD* (1.5k+ estrellas en GitHub), permite explícitamente que los módulos referencien los contratos públicos de otros módulos. En nuestro contexto, extendemos este principio: la superficie pública incluye *Domain* (entidades, contratos) y *Application* (handlers con sus *Command DTOs*), mientras que repositorios e infraestructura permanecen *internal*.

IX. ENMIENDAS PROPUESTAS

Las enmiendas preservan los principios fundamentales del Manifiesto (Clean Architecture, reutilización, calidad, gobernanza) mientras alinean las reglas con la literatura de la industria.

IX-A. Enmienda 1: Separación Asimétrica CQRS

Enmienda 1

La separación estricta *Domain*→*Interface*→*Infrastructure* aplica obligatoriamente para **operaciones de escritura** (Commands). Para **operaciones de lectura** (Queries), los query handlers pueden acceder a infraestructura directamente.

Justificación: Microsoft [12], Young [11], Bogard [13], Vernon [3].

Dentro de un Command, se permiten dos tipos de lectura:

- **Carga de aggregate vía repositorio**: cuando se va a mutar la entidad.

- **Query de soporte vía IQueryable:** validaciones simples, verificaciones de existencia.

IX-B. Enmienda 2: Dependencias Explícitas y Direccionales

Enmienda 2

Reemplazar “aislamiento total” por tres reglas verificables:

R1 — Propiedad exclusiva de escritura. Solo el módulo dueño de una entidad puede crearla, modificarla o eliminarla.

R2 — Referencias de dominio explícitas. Un módulo puede referenciar entidades y value objects del dominio de otro módulo. La dependencia debe ser unidireccional. Las referencias circulares están prohibidas.

R3 — Internos encapsulados. Los repositorios e infraestructura de un módulo son *internal*. La superficie pública es: Domain (entidades y contratos) y Application (handlers con sus Command DTOs validados), permitiendo la reutilización de lógica de negocio vía *Execute*.

Justificación: Evans [2] (*Shared Kernel*), Grzybek [14], Vernon [3].

Tabla II
MATRIZ DE DEPENDENCIAS PROPUESTA

Origen	Destino	¿Perm.?
Modulo.Domain	Otro.Domain (entidades, VOs)	Sí
Modulo.Application	Otro.Domain	No
Modulo.Application	Otro.Application (handlers)	Sí
Modulo.Infrastructure	Otro.Infrastructure	No
Cualquier módulo	Otro (Command DTOs)	Sí

IX-C. Enmienda 3: Comunicación Inter-Modular Atómica

Enmienda 3

Permitir que un handler orqueste operaciones de múltiples módulos dentro de la misma transacción, bajo estas condiciones:

1. La comunicación usa **Command DTOs validados** (FluentValidation).
2. Solo el handler orquestador controla la transacción.
3. Los módulos invocados ejecutan lógica sin comitear.
4. Las dependencias son **unidireccionales** y documentadas.

Justificación: Vernon [3] (*Application Service*), Jovanović [15], patrón existente en ACIDE.

X. RESUMEN COMPARATIVO

XI. CONCLUSIONES

El Manifiesto ACIDE tiene una visión estratégica sólida y mecanismos de gobernanza maduros. Su estructura de capas es consistente con Clean Architecture y DDD. El modelo Snapshot & Fork es pragmático y efectivo para una consultora.

Tabla III
RESUMEN DE ENMIENDAS PROPUESTAS

#	Regla Actual	Enmienda
1	Separación estricta unidireccional para lecturas y escrituras	Estricta para escrituras; lecturas usan IQueryable
2	Aislamiento total, cero cross-import	Dependencias explícitas, direccionales, no circulares
3	Comunicación solo por eventos o shared	Command DTOs validados; transacción única

Sin embargo, las reglas de comunicación inter-modular importan restricciones de sistemas distribuidos que no aplican al contexto real de ACIDE. Esto genera ocho antipatrones documentados que erosionan la productividad y la integridad técnica del sistema.

Las tres enmiendas propuestas no debilitan el Manifiesto. Lo alinean con:

- La documentación oficial de Microsoft para CQRS [12].
- El *Blue Book* de Evans [2] y el *Red Book* de Vernon [3].
- Clean Architecture de Martin [4].
- La implementación de referencia de Grzybek [14].
- El patrón existente en el propio código de ACIDE.

Los principios fundamentales —Clean Architecture, reutilización vía Garaje, calidad, gobernanza— se mantienen intactos. Las enmiendas solo ajustan las reglas de aislamiento al contexto real de un monolito modular.

REFERENCIAS

- [1] E. Romario Huayhua Paco, “Manifiesto de Arquitectura Evolutiva y Gestión de Activos Digitales (ACIDE);” ACIDE, versión 19/03/2026.
- [2] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003.
- [3] V. Vernon, *Implementing Domain-Driven Design*. Addison-Wesley, 2013.
- [4] R.C. Martin, *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Prentice Hall, 2017.
- [5] R.C. Martin, *Agile Software Development, Principles, Patterns, and Practices*. Prentice Hall, 2003.
- [6] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 1st ed. O’Reilly, 2015.
- [7] S. Newman, *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. O’Reilly, 2019.
- [8] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- [9] M. Fowler, *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, 1999.
- [10] M. Fowler, “Monolith First,” *martinfowler.com*, Jun. 2015. [Online]. Available: <https://martinfowler.com/bliki/MonolithFirst.html>
- [11] G. Young, “CQRS Documents,” *cqrs.files.wordpress.com*, Nov. 2010.
- [12] Microsoft, “Microservices architecture e-book: CQRS and DDD patterns,” *learn.microsoft.com*, 2023. [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/architecture/microservices/>
- [13] J. Bogard, “Busting some CQRS myths,” *jimmybogard.com*, Aug. 2012.
- [14] K. Grzybek, “Modular Monolith with DDD,” *github.com/kgrzybek/modular-monolith-with-ddd*, 2019. [Online]. Available: <https://github.com/kgrzybek/modular-monolith-with-ddd>
- [15] M. Jovanović, “Modular Monolith Architecture,” *milanjovanovic.tech*, 2024.
- [16] A. Cockburn, “Hexagonal Architecture,” *alistair.cockburn.us*, 2005.

- [17] S. Tilkov, "Don't start with a monolith," *martinfowler.com*, Jun. 2015.
- [18] J. Spolsky, "The Law of Leaky Abstractions," *joelonsoftware.com*, Nov. 2002.
- [19] F.P. Brooks, "No Silver Bullet—Essence and Accident in Software Engineering," *Computer*, vol. 20, no. 4, pp. 10–19, Apr. 1987.
- [20] C.W. Krueger, "Software reuse," *ACM Computing Surveys*, vol. 24, no. 2, pp. 131–183, Jun. 1992.
- [21] K. Pohl, G. Böckle, and F.J. van der Linden, *Software Product Line Engineering*. Springer, 2005.
- [22] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010.
- [23] W.J. Brown, R.C. Malveau, H.W. McCormick, and T.J. Mowbray, *AntiPatterns: Refactoring Software, Architectures, and Projects in Crisis*. Wiley, 1998.
- [24] K. Beck, *Extreme Programming Explained: Embrace Change*, 2nd ed. Addison-Wesley, 2004.
- [25] M. Morisio, M. Ezran, and C. Tully, "Success and failure factors in software reuse," *IEEE Trans. Softw. Eng.*, vol. 28, no. 4, pp. 340–357, Apr. 2002.